

WasmFX in V8

01 Where we are

02 Design

03 Performance

04 Next Steps

05 Discussion

01

Where we are

Prototype

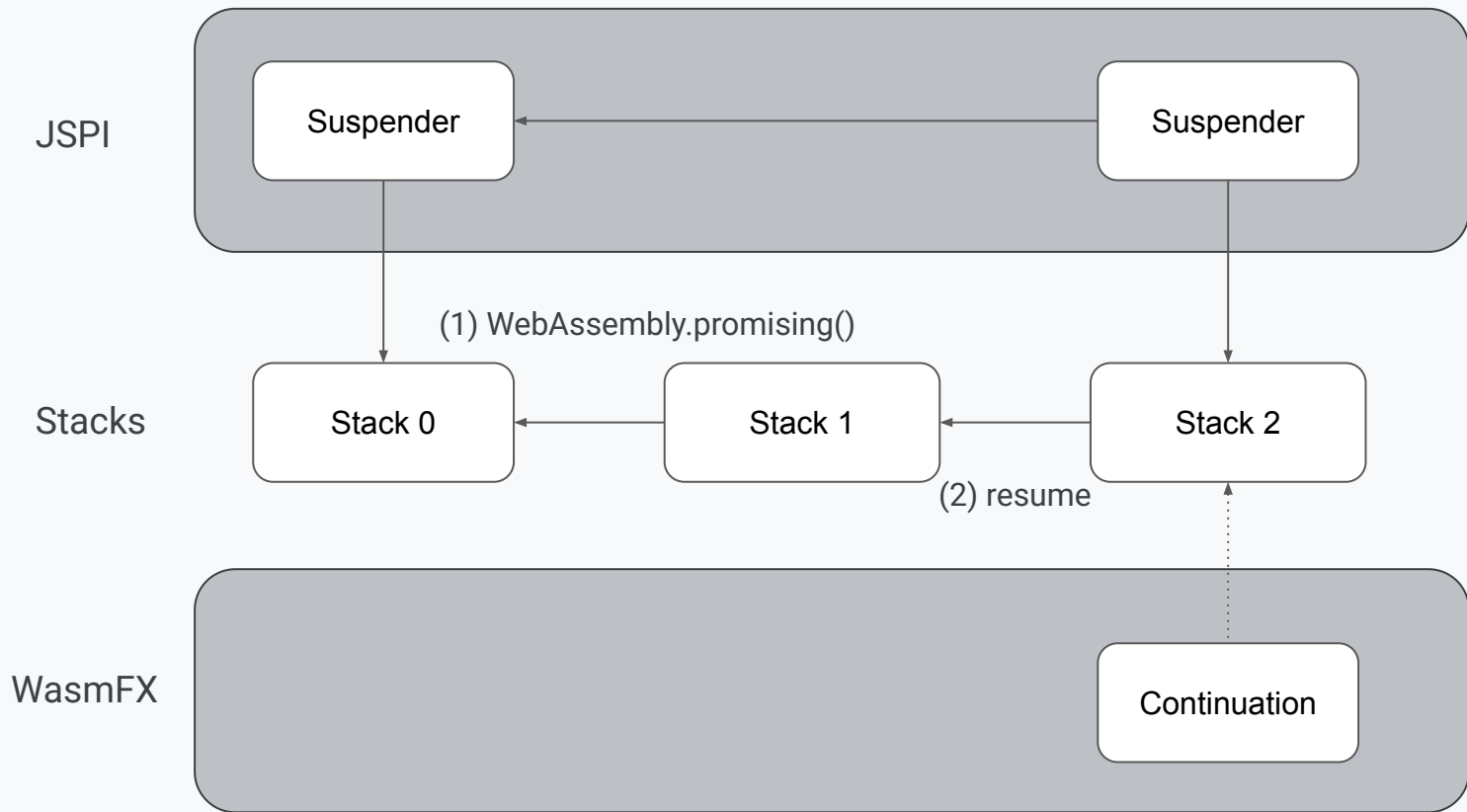
- Supports all instructions
- Optimizing tier only
 - Slow startup
 - No debugging support

02 Design

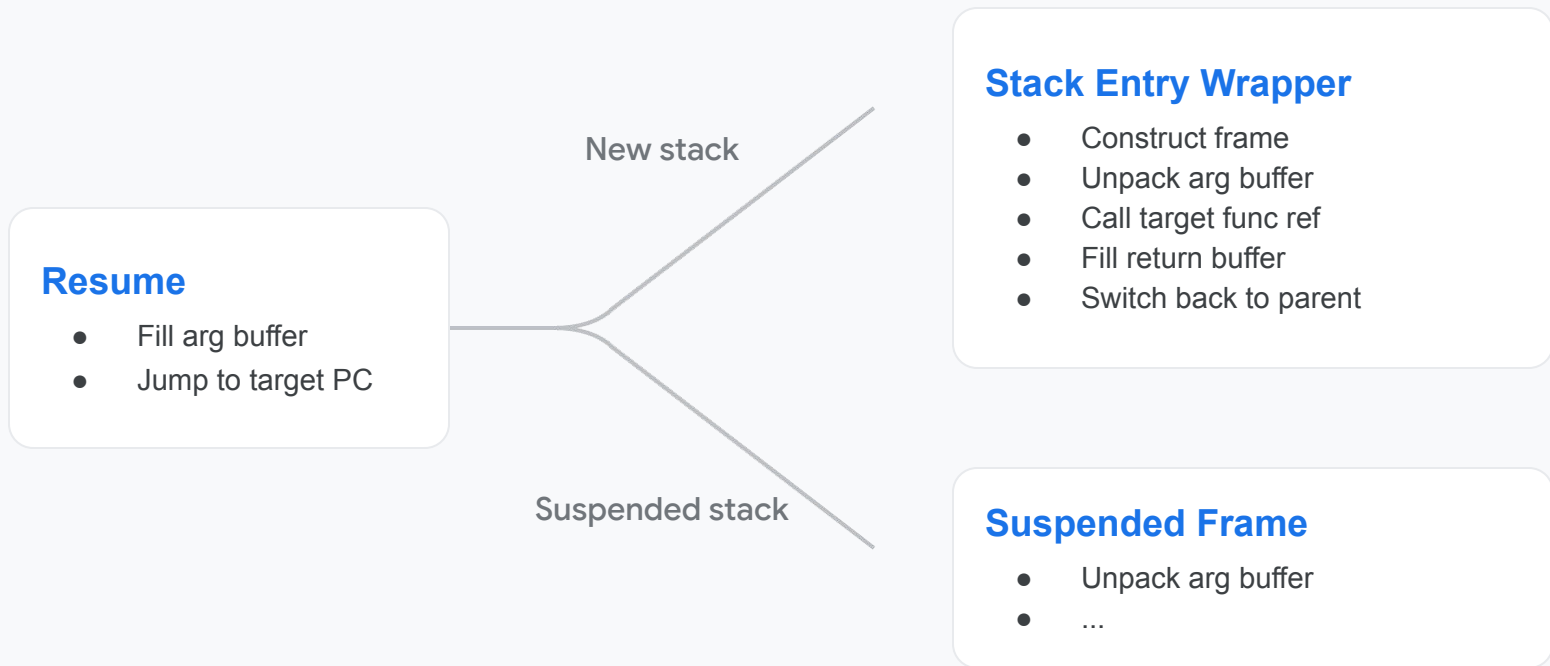
Stacks

- Reusing JSPI stack infrastructure
- By default: 1MB stacks, not growable
- Stack pool of retired stacks
- Segmented growable stacks behind a flag
- Global vector of all stacks, scanned synchronously
- Linked list of stacks to represent nested prompts (JSPI/WasmFX)

WasmFX + JSPI



Resuming



Effect Handlers

- IR representation similar to exception handlers
- Builtin call has three kinds of control-flow successors:
 - Normal return
 - Exception handler
 - Effect handler
- Stored as a vector of (return address, tag index, handler offset) triplets in the code metadata
- On suspend: iterate stacks, lookup code object, and search for a matching return address + tag in the vector

Parameters

Before introducing `cont.bind`:

- Stack-allocated arg buffer in the source stack
- Arguments stored from left to right
- Pass pointer to the buffer across stack switches
- Unpacked immediately by target, the buffer does not need to be scanned

Changes to support `cont.bind`:

- Allocate arg buffer in the suspended stack
- Store arguments in reverse order, so that binding does not shift the offsets
- Store signature and number of bound arguments in the metadata
- Scan bound arguments during GC

V8 Heap Sandbox

The main escape surface is the pointer from the on-heap continuation object to the off-heap stack object.

The sandbox offers a **type-safe external pointer table** for such pointers. So under the sandbox threat model, the continuation's index into the table can be corrupted, but it must point to a valid stack.

Under this assumption, on stack switch, we check that the target stack:

- Is in the expected state (Active/Inactive/Suspended/Retired)
- Has the expected signature

This is necessary and sufficient to prevent escaping the sandbox.

03

Performance

Performance

Microbenchmark: sum of first N integers

- Stack switching generator + local counter
- Function call + global counter

Direct call	Call ref	Stack switching
2.4ns/iteration	4.2ns/iteration	53ns/iteration

(CPU: AMD Ryzen Threadripper Pro)

Suspend is significantly slower than resume

Performance

Kotlin coroutine benchmark¹

	State machine	Stack switching
coroutineIteration	34.1ms/op	109.2ms/op
coroutineRecursion	339.4ms/op	773.4ms/op

Kotlin compose benchmark: 1-1.8x slowdown

¹ <https://github.com/Kotlin/kotlin-wasm-benchmarks/blob/main/src/commonMain/kotlin/macroBenchmarks/Coroutines.kt>

04

Next steps

Next Steps

- Gather and investigate benchmarks
- Improve peak performance
 - Optimize handler search
 - Avoid code cache lookup
 - ...
 - Inline builtin calls?
- Stabilize growable stacks
 - Blocked by an issue with tail calls
- Port instructions to Liftoff (baseline/debug tier)
- DevTools support
- ...

Flags

- `--experimental-wasm-wasmfx`
- `--experimental-wasm-growable-stacks`
- `--wasm-stack-switching-stack-size=<kb>`

05 Discussion